ROBIN SCHROER

TRACING RUST AT SCALE

ABOUT ME

- Staff engineer at KOMOJU
  - Payment processing
  - Mostly Ruby, some Rust
- Working in core platform engineering
  - Shared libraries, observability, lots of other stuff
- Rust user since 1.36
  - I remember `try!`

GOALS

- Get an overview of tracing
- Learn more about tracing than you ever wanted to
- Instrument your own distributed Rust systems

AGENDA

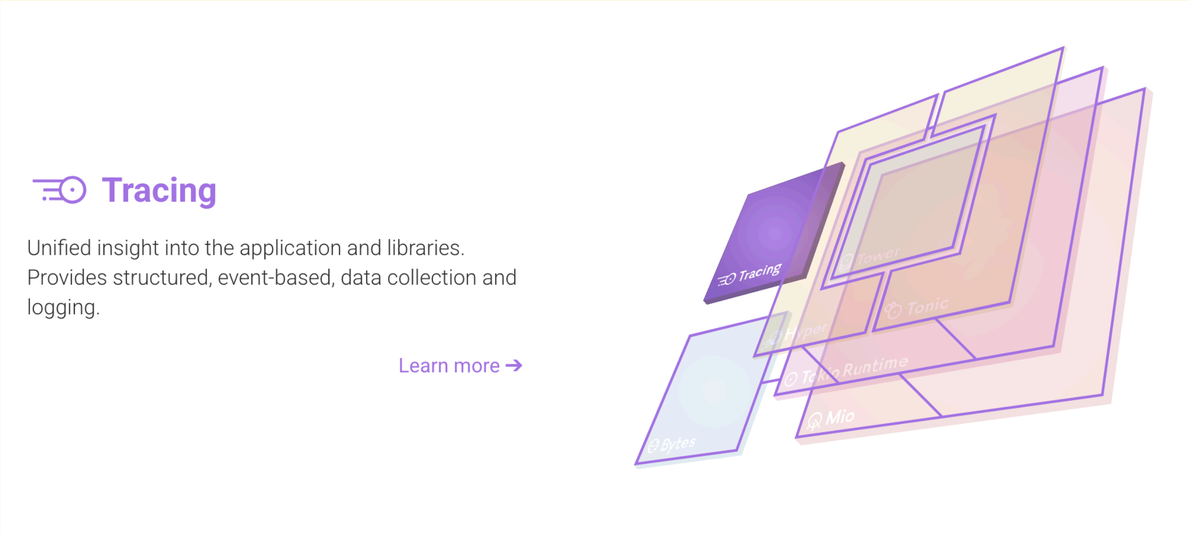
- Three parts
  - Tracing basics
  - Tracing implementation details
  - Practical solutions
- Questions at the end please
- Slides are numbered
- Slides are also at blog.sulami.xyz/media/tracing-rust-at-scale.pdf

PART 1

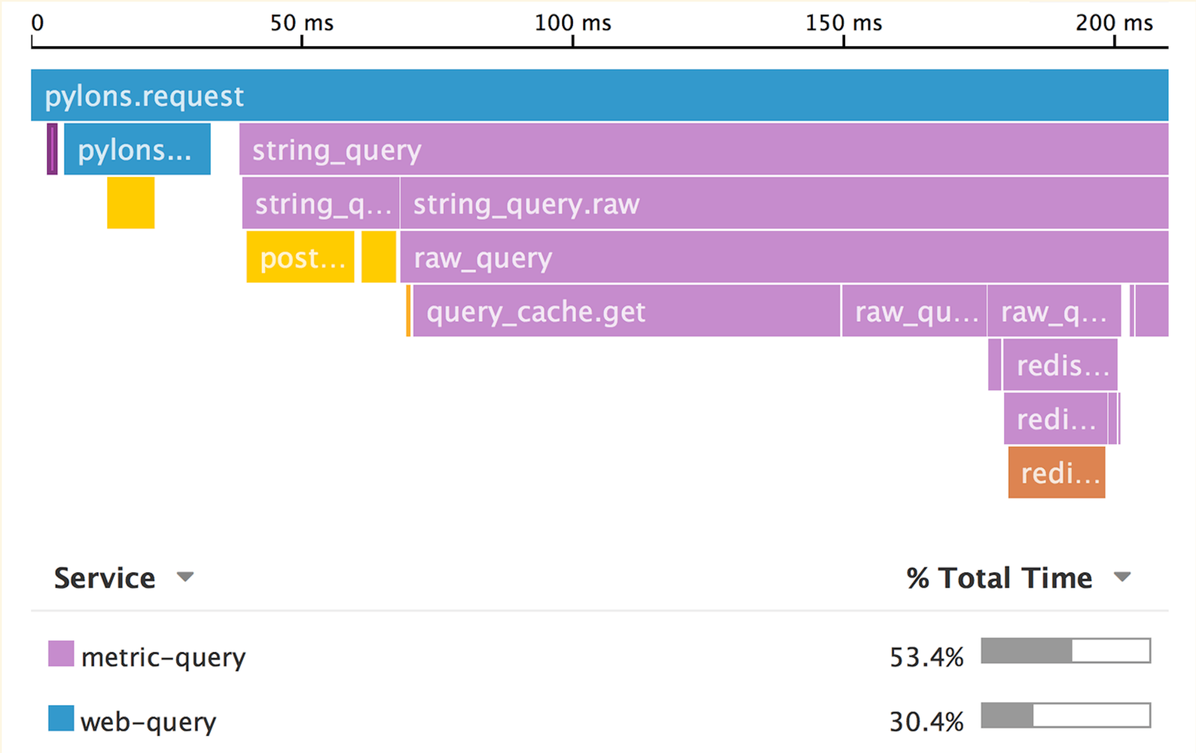
SETTING UP

QUICK DISTRIBUTED TRACING PRIMER

NAMING IS HARD



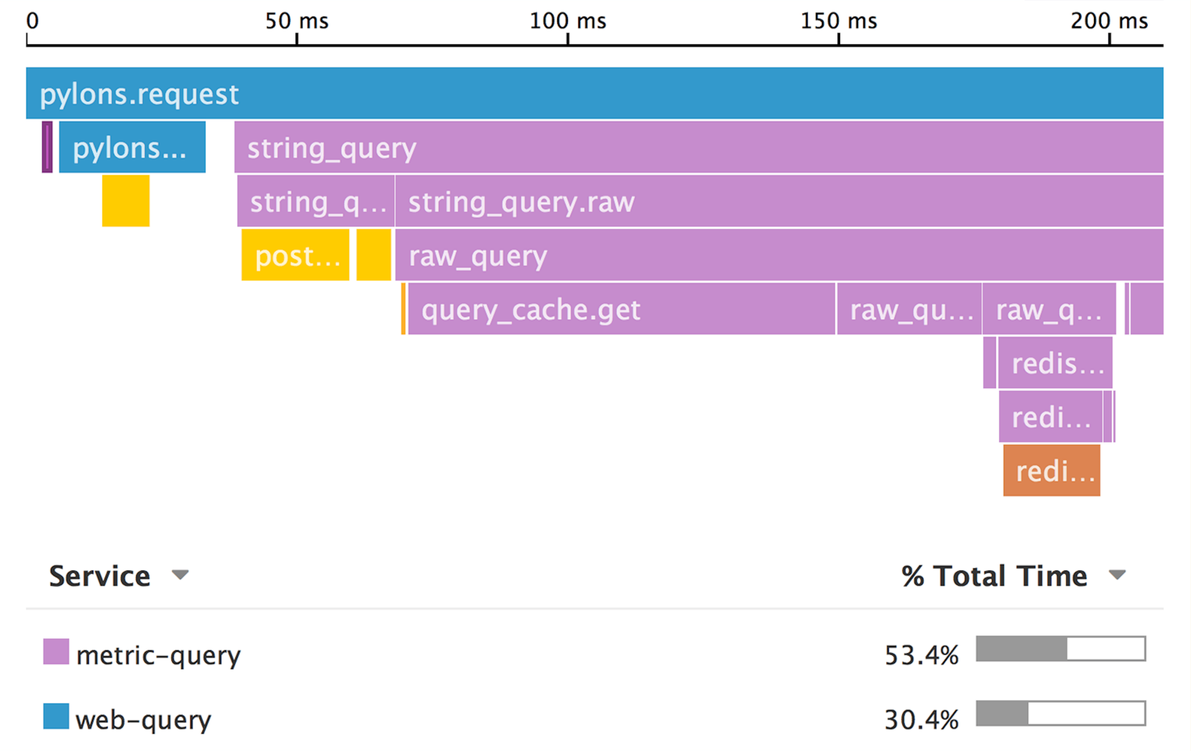
Tokio `tracing` library



Tracing

TRACING

- This is a trace
- Commonly visualised as flame chart
  - Not a flame graph
  - x-axis has passage of time
- Has some spans
  - Tree structure, root at the top
  - Different colour-coded components



TRACES

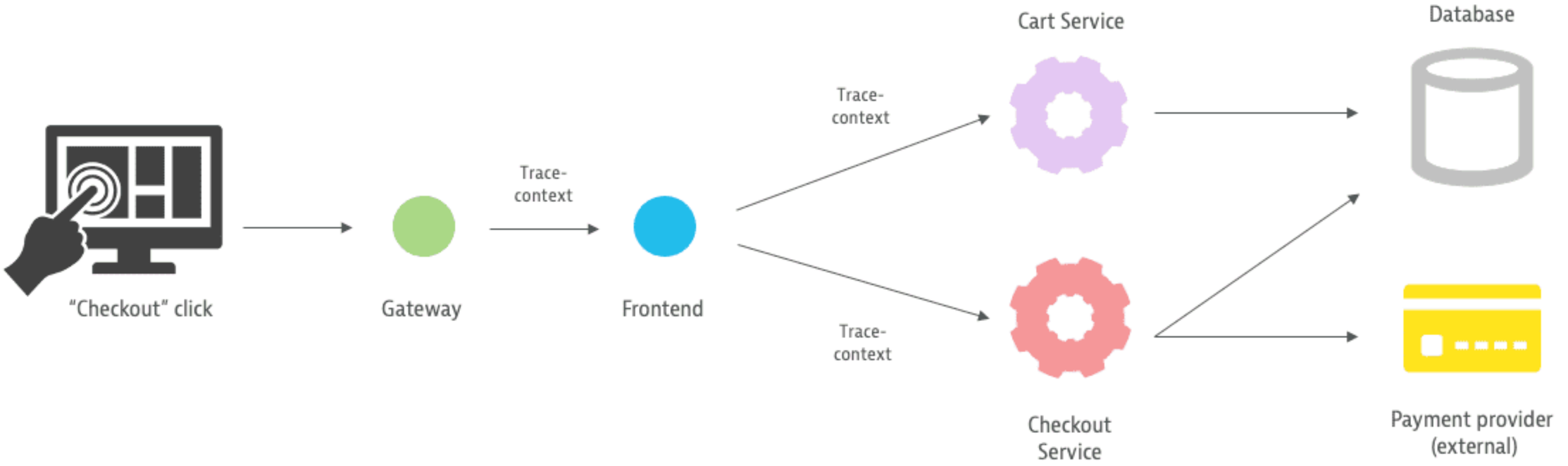
- Are one operation or interaction, e.g. one HTTP request or message handled
- Have a trace ID (64- or 128-bit)
- Contain one or more spans forming a tree

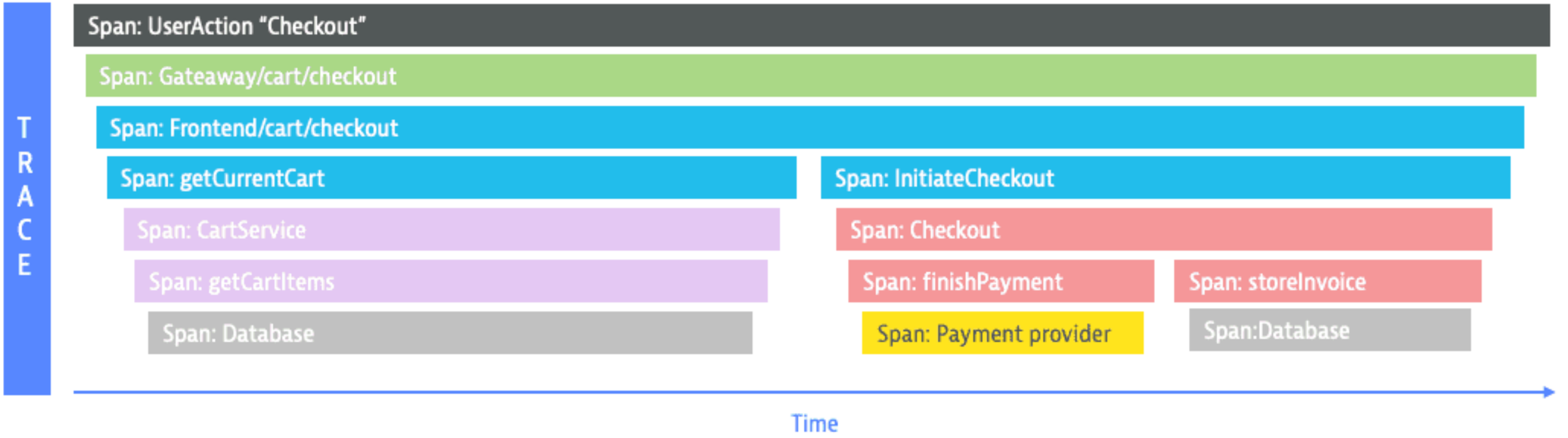
SPANS

- Have a span ID (64-bit)
- Belong to a trace via the trace ID
- Have either a parent span ID or are a root span
  - Technically multiple roots are possible, but let's not go there
- Have a start time and end time (or start and duration)
- Can have arbitrary metadata tags

DISTRIBUTED TRACING

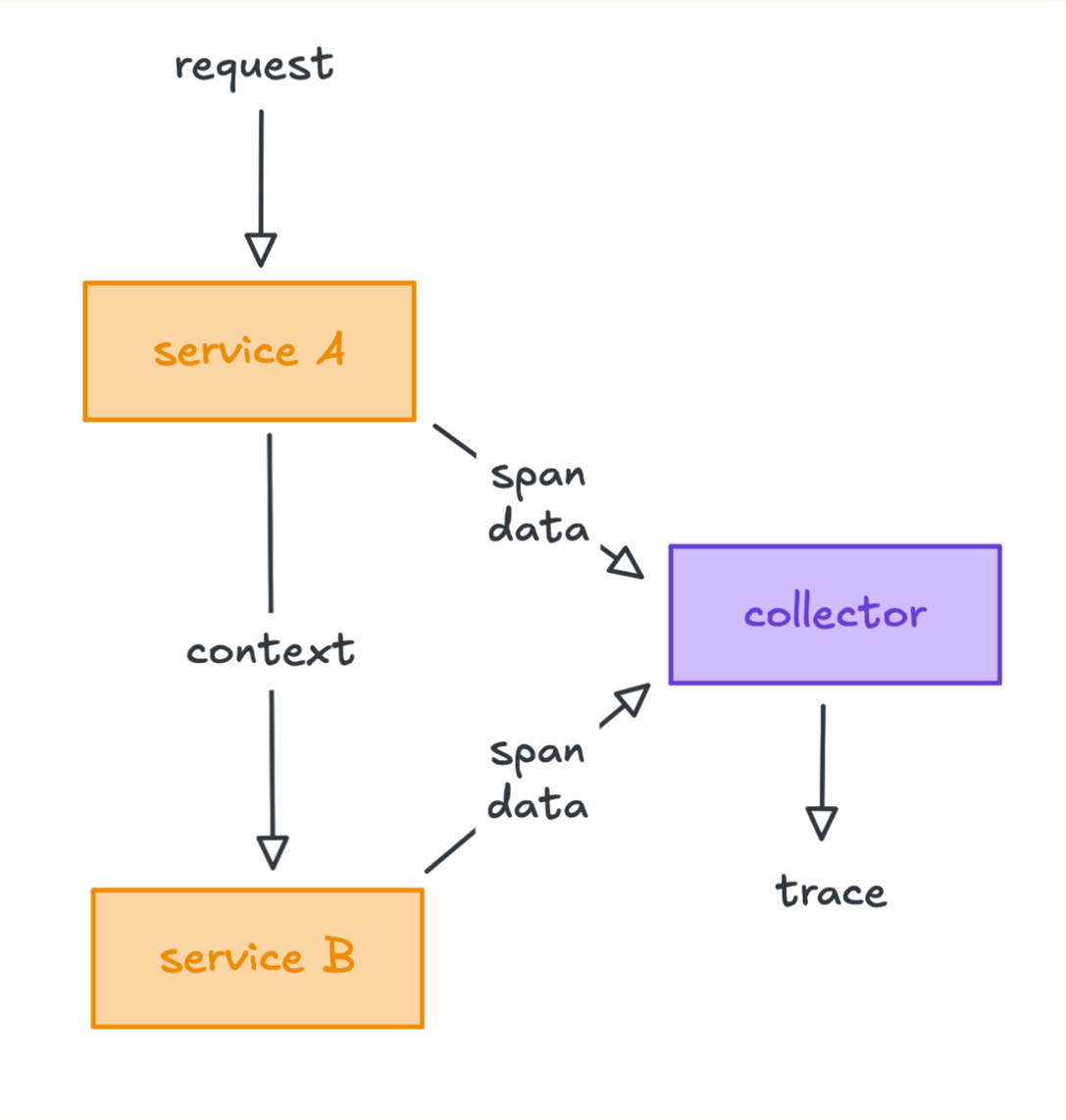
- Non-distributed tracing is what the `tracing` library does
  - Spans create nested context
  - Each layer captures additional data as span tags, e.g. function arguments
- Once we cross a boundary into a different system, we lose the context
- Distributed tracing holds onto the context across boundaries


Cart Service
Database
Trace-
context
Trace-
context
"Checkout" click
Gateway
Frontend
Trace-
context
Checkout
Service
Payment provider
(external)


TRACE
Span: UserAction "Checkout"
Span: Gateaway/cart/checkout
Span: Frontend/cart/checkout
Span: getCurrentCart
Span: InitiateCheckout
Span: CartService
Span: Checkout
Span: getCartItems
Span: finishPayment
Span: storeInvoice
Span: Database
Span: Payment provider
Span:Database
Time

PRESERVING CONTEXT

- Passing along entire spans would be prohibitively expensive
  - Each component submits trace data to a collector
  - Only IDs are passed between components
  - Collector stitches back together the entire trace



WHAT YOU GET FROM DISTRIBUTED TRACING

- End-to-end visibility of request flows
- Easily identify latency bottlenecks
- Automatic service maps
- Deployment tracking
- Almost like a debugger attached to production

PART 2

EXPLORATION

SUBMITTING TRACE DATA

OPENTELEMETRY

- CNCF-backed framework of APIs, SDKs, and tools to work with o11y data
- Platform-agnostic
- Traces, logs, metrics, and profiling through OTLP
- Usually some amount of assembly required
- Generally no support for vendor-specific features

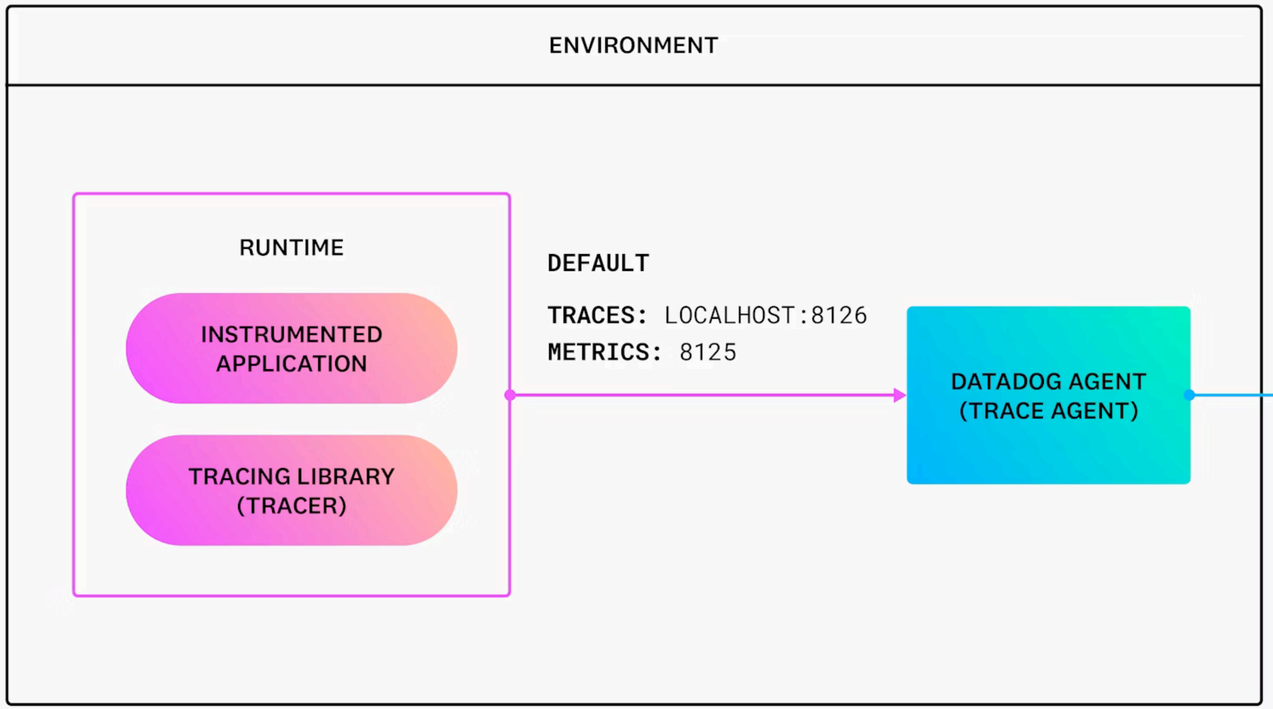
COMMON TRACING PROVIDERS

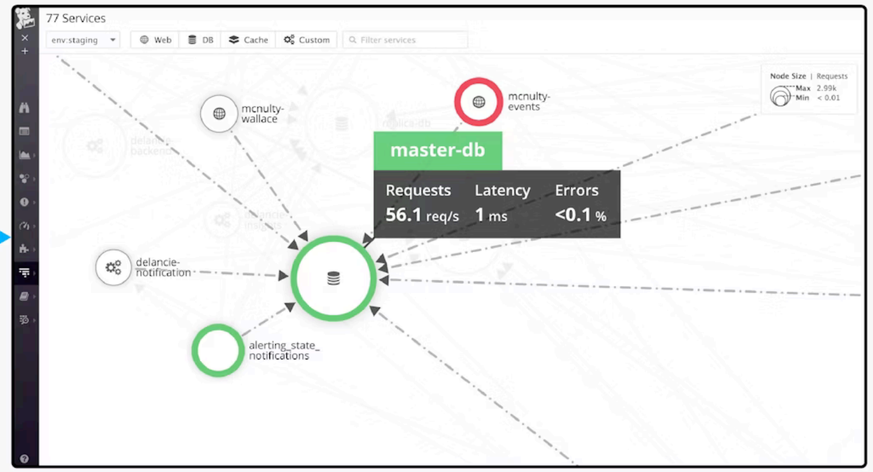
- Datadog
- Honeycomb
- Sentry
- Jaeger
- Grafana
- Kibana

DATADOG

- Does all kinds of observability, not just tracing
- Can correlate all kinds of data from different sources
- Agent-based architecture
  - Traces submitted to agent
  - Agent also pulls data from other sources
- SDKs for basically all major languages and platforms, except Rust


APPLICATION+ TRACER
DATADOG AGENT
BACKEND/UI


ENVIRONMENT
RUNTIME
INSTRUMENTED APPLICATION
TRACING LIBRARY (TRACER)
DEFAULT
TRACES: LOCALHOST:8126
METRICS: 8125
DATADOG AGENT (TRACE AGENT)


77 Services
env:staging
Web
DB
Cache
Custom
Filter services
mcnulty-wallace
mcnulty-events
master-db
Requests
Latency
Errors
56.1 req/s
1 ms
<0.1 %
delancie-notification
alerting_state_notifications

SAMPLING

- Keeping all trace data is usually not feasible
- Sampling can occur on any step of the journey
- You want entire traces to be sampled
- Datadog has 15-minute retention for all trace data
- Datadog derives trace metrics from the entire trace data
- No client-side sampling, just retention filters

DATADOG AGENT INGRESS

- Agent can act as OTLP collector if configured
  - Requires OpenTelemetry data
  - More work to get the data right because of differences in data models
- Datadog-specific trace API
  - MessagePack over HTTP, port 8126
  - Several API versions available
  - Specification available in agent codebase

DATADOG AGENT TRACE API

- v0.4 trace chunks, arrays of spans
- v0.5 deduplicates strings into a dictionary sent alongside the spans
- v1.0 uses a new format, the tracer payload, for more features
- Protobufs are available for all formats
- github.com/DataDog/datadog-agent/blob/main/pkg/trace/api/version.go

DATADOG AGENT V0.4 TRACE API

- Array of spans, sorted by trace
- Some important tags are top-level fields, the rest in maps
  - Strings and numbers separated for encoding
- github.com/DataDog/datadog-agent/blob/main/pkg/proto/datadog/trace/span.proto

```
struct Span {
    trace_id: u64,
    span_id: u64,
    parent_id: u64,
    start: i64,
    duration: i64,
    name: String,
    service: String,
    r#type: String,
    resource: String,
    meta: HashMap<String, String>,
    metrics: HashMap<String, f64>,
}
```

COLLECTING TRACE DATA

MAJOR RUST TRACING LIBRARIES

- No first-party Datadog SDK at his time
- Tokio `tracing`
- `opentelemetry`
- `fastrace`

TOKIO TRACING

- The only one with meaningful library support
- Powerful `#[instrument]` proc macro
- Flexible output system with layers & subscribers
  - Many existing subscribers: `tracing-chrome`, `tracing-tracy`, `tokio-console`
  - No direct way to export trace data to Datadog
- Requires all span tags to be declared up-front

OPENTELEMETRY

- Can be used directly instead of `tracing`
- Can subscribe to `tracing` data
- Can export to Datadog in two ways
  - OTLP via `opentelemetry-otlp`
  - Trace API via `opentelemetry-datadog`
- Requires keeping a lot of libraries from different sources in sync
- Data model differences lead to inconsistencies

FASTRACE

- Pretty new, supposedly fast, but much more limited in features
- Subscriber for `tracing` exists
- Trace API export via `fastrace-datadog`
- Somewhat brittle in our experience, especially distributed context
  - Maybe check back in a year

TRACE CONTEXT PROPAGATION

DISTRIBUTED TRACING

- Once several services are involved, we need to propagate trace context
- Trace ID, parent span ID, and sampling decision
- Several existing standards for HTTP headers
  - Context can be transparently injected and extracted in middleware
  - Also works for gRPC, GraphQL, and similar HTTP-based protocols
- Other protocols like WebSockets or message queues are usually custom in-band protocols

EXISTING TRACE CONTEXT HTTP HEADER STANDARDS

- W3C Trace Context
- Datadog headers
- B3 (single & multi)
- W3C Baggage

RELEVANT TRACE CONTEXT HEADER STANDARDS

- W3C Trace Context
  - `traceparent` & `tracestate` HTTP headers
  - w3c.github.io/trace-context/
- Datadog trace headers
  - `x-datadog-trace-id`, `x-datadog-parent-id`, etc.
  - docs.datadoghq.com/tracing/trace_collection/trace_context_propagation/

W3C TRACE CONTEXT

- `traceparent` header
  - `<version>-<trace-id>-<parent-id>-<trace-flags>`
  - only version is `00`
  - trace ID is up to 128-bit
  - only flag is `01` for sampled
  - all lower-case hex
  - example: `00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01`
- `tracestate` header is for extra information, we don't need it

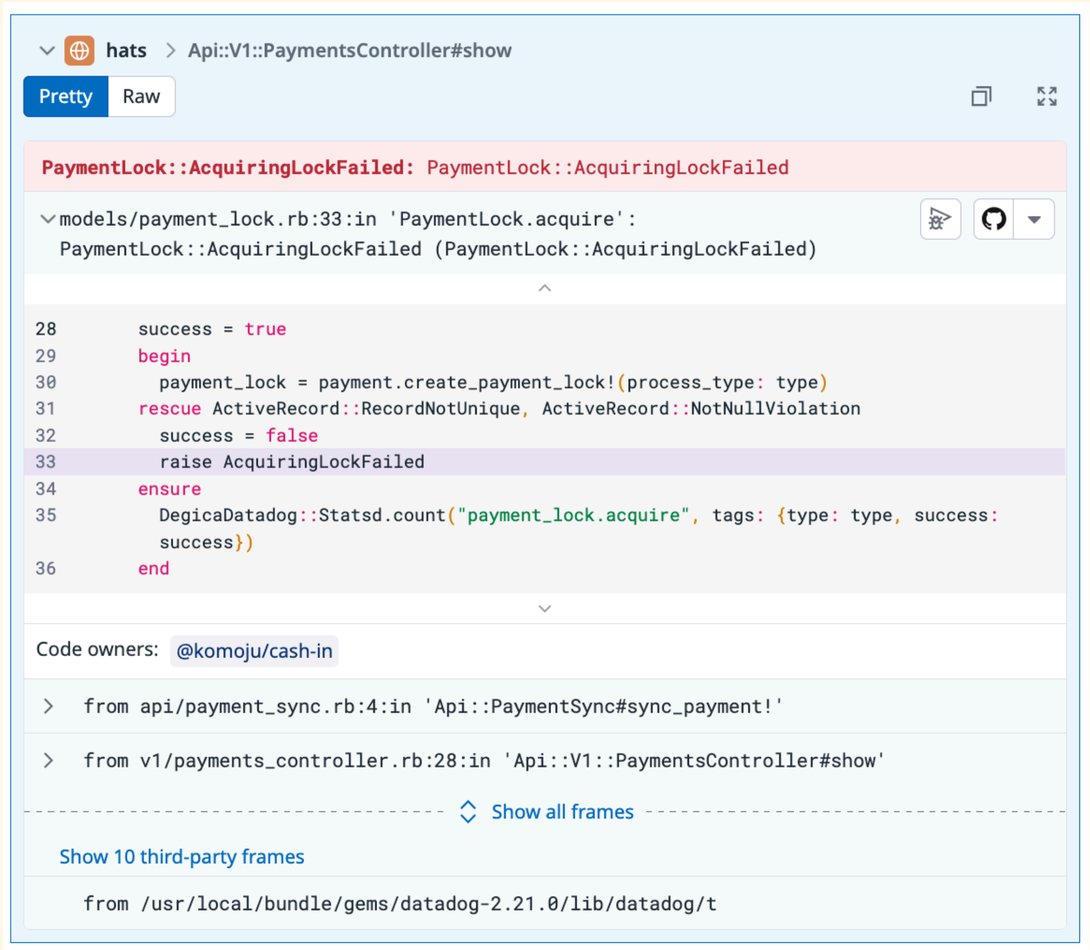
SEMANTIC CONVENTIONS

DATADOG SPAN TAG SEMANTICS

- Datadog expects certain span tags to function as intended
  - docs.datadoghq.com/standard-attributes/?product=apm
- These are needed on every single span:
  - Unified service tagging (UST) uses `service`, `env`, and `version`
  - `operation` and `resource` are used for grouping
  - `span.kind` (server, client, internal, ...) has search indexing implications

MORE DATADOG SPAN TAG SEMANTICS

- Remote service calls might use `peer.service` or `peer.hostname`
- Errors have their own semantics
  - Needed for error tracking as well
- HTTP requests have their own semantics
- DB queries have their own semantics



SECRET SPAN TAGS

- These tags are also required, but not really documented anywhere
  - `_dd.top_level` - special marker for service-entry spans
  - `_dd.measured` - enables trace metric calculation (latency, etc.)

CORRELATION

A MORE COMPLETE PICTURE

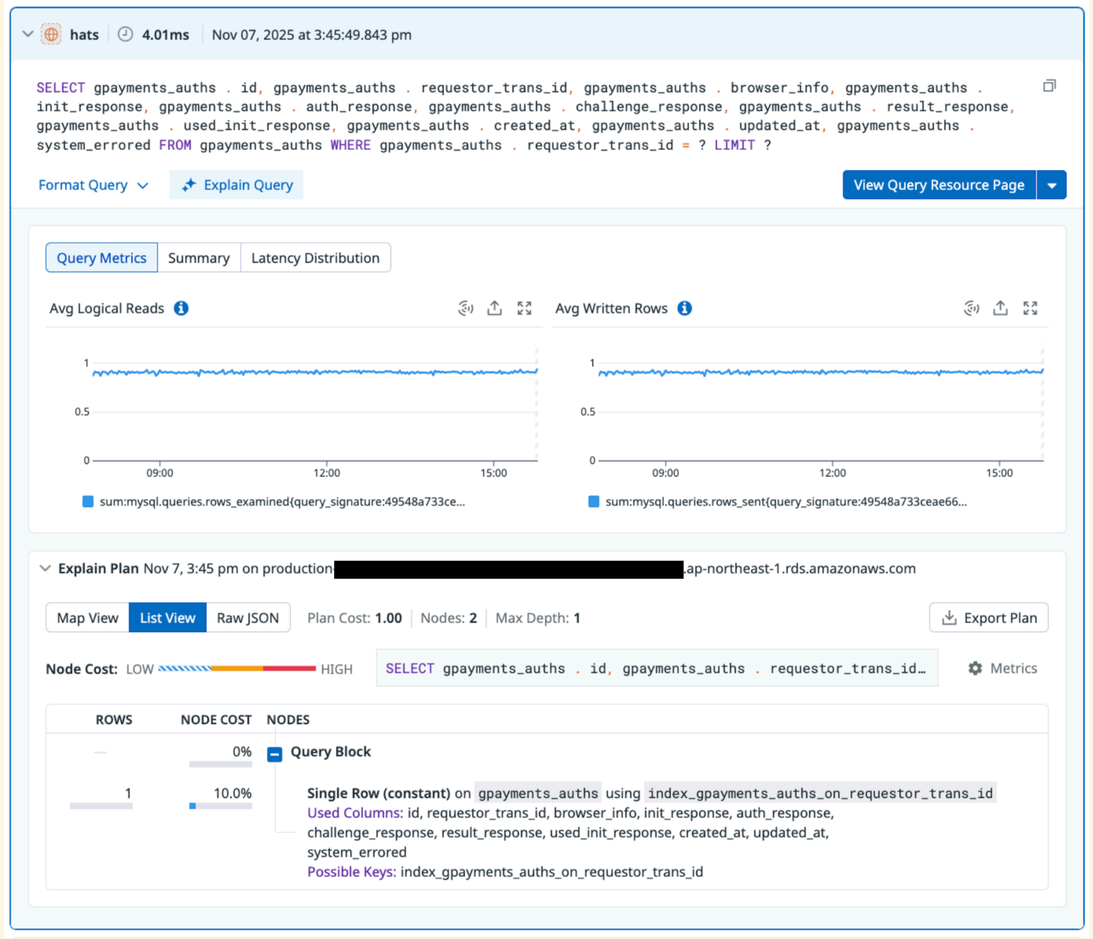
- Lots of other data is coming through different channels
- Datadog can correlate the data to tracing if we give it enough information
- We basically have to pass trace- and span-IDs everywhere
- More semantic conventions

LOGGING

- Correlation enables logs for a trace and trace for a log line
- Logs generally go to stdout, depending on the platform
- JSON with some undocumented semantics
  - Message field
  - Trace- and span-ID as fields for correlation
  - Other tags use `fields.<name>`
  - Different enough that `tracing-subscriber`'s JSON output doesn't work well
  - github.com/open-schnick/datadogformattinglayer implements the format as a `tracing` layer using `opentelemetry`

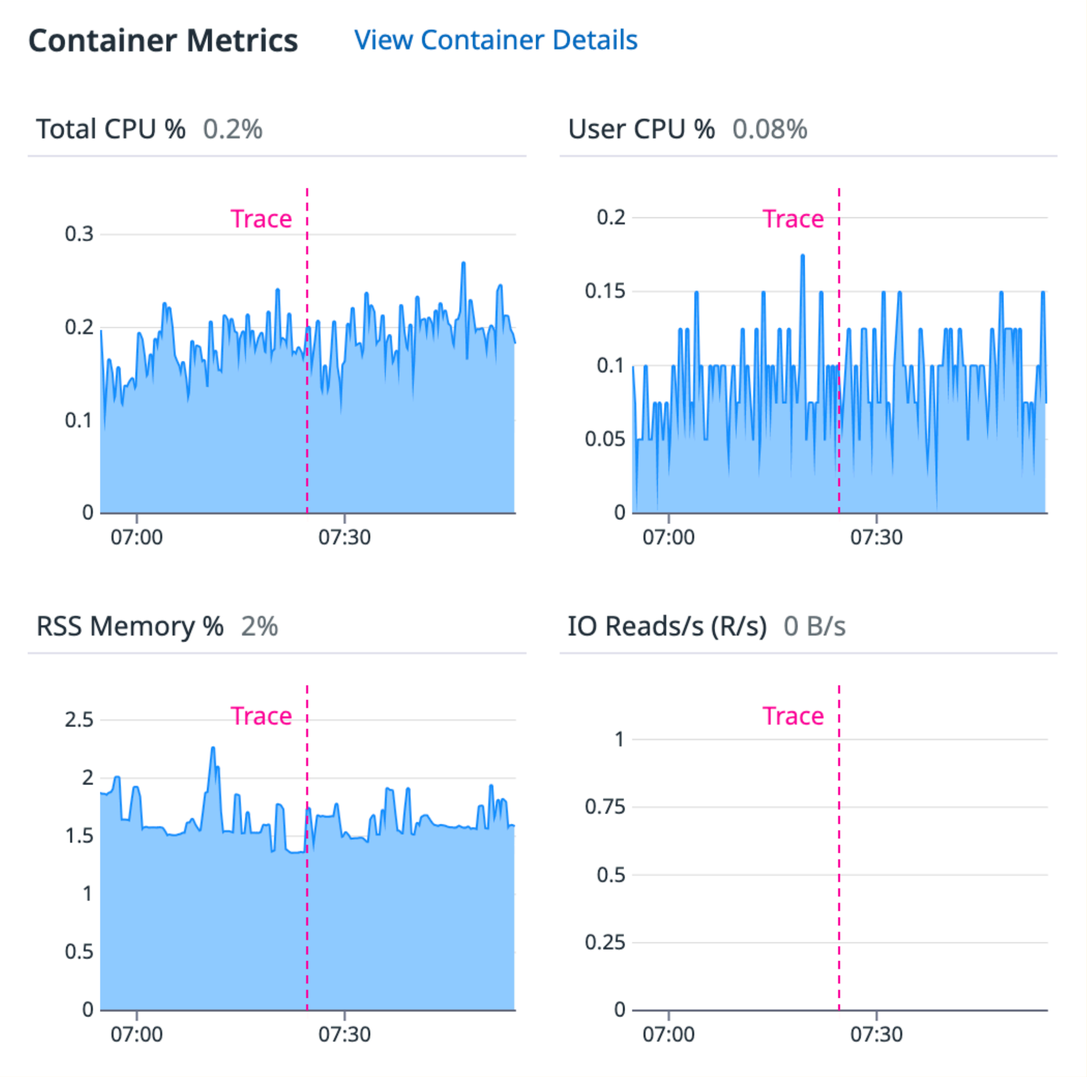
DATABASE QUERIES

- Correlation provides inline stats and explain plans for traces, as well as traces for a query
- Agent collects data directly from database
- Semantic tagging
  - `span.type = "sql"`, `db.statement`, etc.
- Prefix comments with trace- and span-ID tags on every query



INFRASTRUCTURE METRICS

- Correlation enables metrics for a trace and traces for a host
- Agent can collect infrastructure data, depending on platform
- No span tag for that correlation
  - Trace API accepts `Datadog-Container-ID` HTTP header



PART 3

CONVERGENCE

PUTTING IT ALL TOGETHER

COMMON STACK

- `tracing`
- `tracing-opentelemetry`
- `opentelemetry`
- `opentelemetry-datadog`
- `datadog-formatting-layer`
- Some more supporting libraries
- Lots of custom code for semantic tagging, context propagation, etc.

PAIN POINTS

- 100+ lines of code just to get trace data to Datadog at all, several hundred for decent results
- 7+ libraries from different sources, which all have to stay in sync
  - Remember different crate versions produce different types
  - Some incompatibilities lead to silent failures
  - Some of them are only sort of maintained
- Having to declare `tracing` span tags up-front means every `#[instrument]` annotation includes many boilerplate tags
  - Datadog tagging semantics leak everywhere

KOMOJU-DATADOG

FEATURES

- Few-line tracing setup, mostly use `tracing` as usual
- No dependency on `opentelemetry`
  - Trace data export to Datadog using the trace API
- Datadog-compatible log formatting with correlation
- Axum middleware for request spans and trace context extraction
- Automatic semantic mapping for span tags
  - Configurable system-wide default tag injection
- More features that didn't fit onto this slide

```rust
let o11y_config = komoju_datadog::Config::builder()
    .service("omega-star")
    .env(&settings.o11y_env)
    .version(VERSION)
    .build();
let _tracer = komoju_datadog::tracing::Tracer::new(&o11y_config);
```

SPIN-OFF COMPONENTS

- `tracing-datadog` is just the tracing subscriber
  - Much less opinionated, but more assembly required
  - Trace submission, log formatting, HTTP trace context handling
- `sqlx-datadog` provides an `#[instrument_query]` proc macro
  - Still work in progress, but we run it in production just fine
  - No trace context comment injection yet

GET INVOLVED

- All code is open-source
  - github.com/komoju/komoju-datadog
  - github.com/sulami/tracing-datadog
  - github.com/sulami/sqlx-datadog
  - Contributions welcome
- Of course also on crates.io

THANK YOU

FIN

QUESTIONS?

LINKS

- Slides with links at blog.sulami.xyz/media/tracing-rust-at-scale.pdf
  - Or scan that QR code →
- Code
  - github.com/komoju/komoju-datadog
  - github.com/sulami/tracing-datadog
  - github.com/sulami/sqlx-datadog